

EXPERIMENT – 1

1. To implement simple search method using MoveGen(N) and GoalTest(N) where N is concerned node.

Apparatus: Python, NumPy

Theory: Most problem-solving techniques in Artificial Intelligence are goal-oriented, designed to transform an initial state into a desired outcome. These problem spaces are typically represented using graphs or state-space decision trees. Before exploring specific graph traversal strategies like Depth-First Search (DFS) or Breadth-First Search (BFS), it is essential to understand the fundamental Simple Search Algorithm. This baseline approach operates on the core principle of Generate-and-Test:

- i. **Generate:** Dynamically generate successor nodes (children) from the current node (parent) using a move generator function (MoveGen()).
- ii. **Test:** Apply a goal test function to validate whether a generated node matches the target outcome.

To maintain state and manage the search process, the algorithm tracks and manipulates nodes using the following parameters and operations:

- i. **Start:** The initial state or root node where the search originates.
- ii. **Goal:** The targeted state used to evaluate whether the search has succeeded.
- iii. **OPEN:** A list that stores all generated nodes that are pending expansion or evaluation.
- iv. **CLOSED:** A list that keeps track of all nodes that have already been expanded or visited, preventing redundant processing and infinite loops.
- v. **MoveGen:** The operator that produces all valid child nodes for a given node.
- vi. **Append:** Inserts newly generated children into the OPEN list.
- vii. **Sort:** Reorders the OPEN list based on predefined criteria or heuristics.

Code:

```
import numpy as np
```

```
class Node:
```

```
    def __init__(self, value: str):
        self.value = value
        self.neighbors: list[Node] = []
    def __repr__(self):
        return self.value
```

```
def move_gen(node: Node) -> list[Node]:
    return node.neighbors
```

```
def goal_test(node: Node, goal: Node) -> bool:
    return node == goal
```

```
def simple_search(start: Node, goal: Node) -> tuple[bool, list[Node]]:
    open = np.array([start], dtype=object)
    closed = np.array([], dtype=object)
```

```
    while len(open) > 0:
```

```

node = open[0]
open = open[1:]
closed = np.append(closed, node)

if goal_test(node, goal):
    return True, closed.tolist()

neighbors = move_gen(node)
for neighbor in neighbors:
    if neighbor not in closed and neighbor not in open:
        open = np.append(open, neighbor)

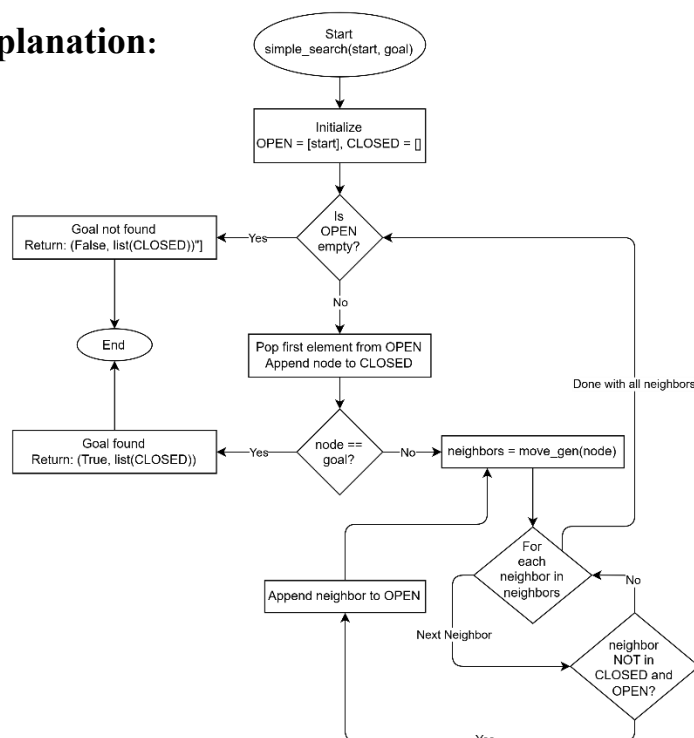
return False, closed.tolist()

if __name__ == "__main__":
    a = Node("A")
    b = Node("B")
    c = Node("C")
    d = Node("D")
    a.neighbors.extend([b, c])
    b.neighbors.extend([a, c, d])
    c.neighbors.extend([a, b, d])
    d.neighbors.extend([c, b])

    status, path = simple_search(a, d)
    print("Search Result:")
    print("\tStart :", a)
    print("\tGoal :", d)
    print("\tGoal Found :", status)
    print("\tVisited:", " -> ".join(str(node) for node in path))

```

Code Explanation:



Output:

```
Search Result:  
  Start : A  
  Goal  : D  
  Goal Found : True  
  Visited: A -> B -> C -> D
```

EXPERIMENT – 2

2.1: To implement Breadth First Search algorithm using Python.

Apparatus: Python

Theory: Breadth-First Search (BFS) is a graph traversal algorithm used to explore the nodes of a graph level by level. It starts from a given node and visits all its adjacent nodes before moving to the nodes at the next level.

In the given implementation, the graph is represented using a **dictionary**, where each key represents a node and its corresponding value contains a list of neighboring nodes. BFS uses the following data structures:

i. Graph: The graph is stored using a Python dictionary. Each node is associated with a list of its directly connected neighboring nodes.

ii. Queue: A deque from Python's collections module is used as the queue. It follows the **FIFO (First In, First Out)** principle. The popleft() operation removes the node at the front of the queue, while newly discovered nodes are added to the rear using append().

iii. Visited: A Python **set** is used to store nodes that have already been discovered. It prevents the same node from being added to the queue multiple times and avoids unnecessary repeated traversal.

iv. Order: A Python **list** is used to maintain the order in which nodes are removed from the queue and visited during the search.

v. Start Node: The node from which the BFS traversal begins. In this program, the starting node is A.

vi. Goal Node: The node that the search attempts to find. In this program, goal node is D. The BFS process begins by placing the start node into the queue and marking it as visited. The node at the front of the queue is then removed and added to the visited order. If it is not the goal, its unvisited neighboring nodes are added to the queue and marked as visited. This process continues until the goal node is found or the queue becomes empty. The use of a queue ensures that nodes are explored level by level, while the visited set prevents repeated processing of nodes.

Code:

```
from collections import deque
```

```
GRAPH = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}
```

```
def breadth_first_search(start: str, goal: str) -> tuple[bool, list[str]]:
    queue = deque([start])
    visited = {start}
    order = []

    while queue:
        node = queue.popleft()
```

```

order.append(node)

if node == goal:
    return True, order

for neighbor in GRAPH[node]:
    if neighbor not in visited:
        visited.add(neighbor)
        queue.append(neighbor)

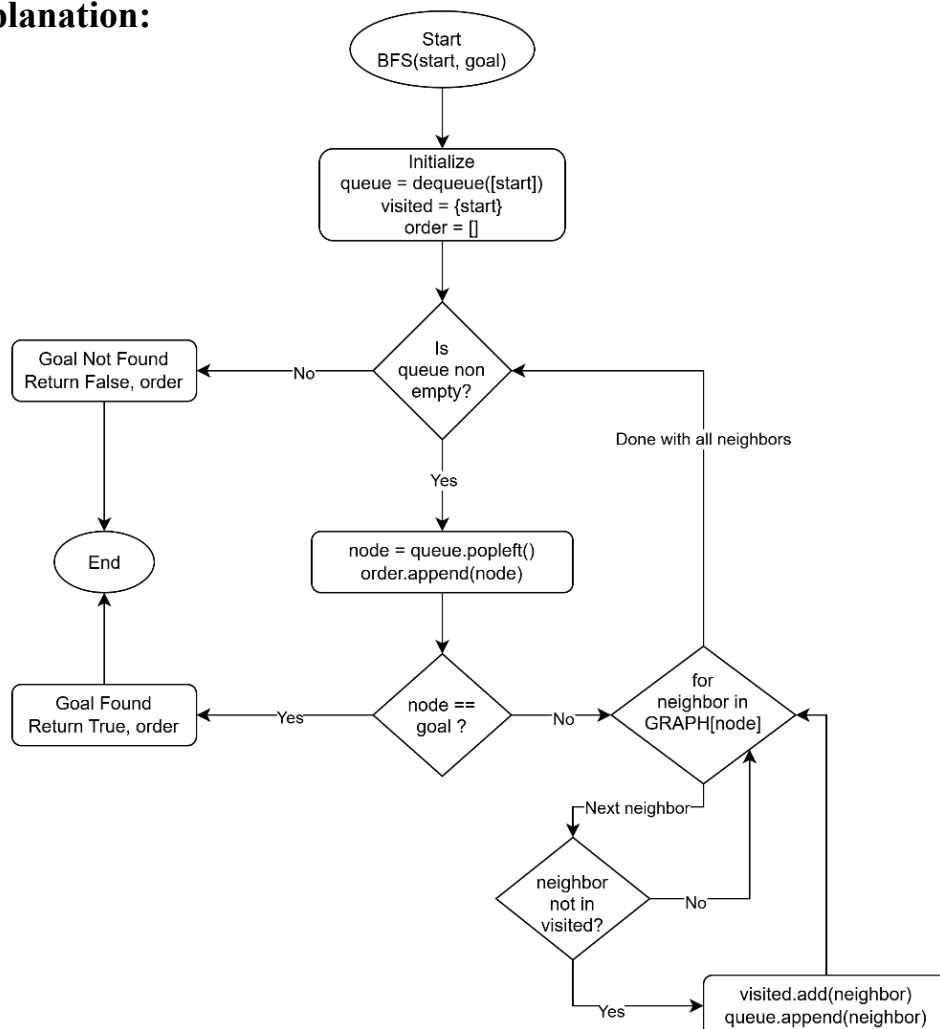
return False, order

if __name__ == "__main__":
    status, path = breadth_first_search("A", "D")

    print("Search Result:")
    print("\tStart : A")
    print("\tGoal : D")
    print("\tGoal Found :", status)
    print("\tVisited:", " -> ".join(path))

```

Code Explanation:



Output:

```
Search Result:  
  Start : A  
  Goal  : D  
  Goal Found : True  
  Visited: A -> B -> C -> D
```

2.2: To implement Depth First Search algorithm using Python.

Apparatus: Python

Theory: Depth-First Search (DFS) is a graph traversal algorithm used to explore the nodes of a graph by going as deep as possible along each branch before backtracking. It starts from a given node and explores an unvisited neighboring node before moving to other neighboring nodes. In the given implementation, the graph is represented using a **dictionary**, where each key represents a node and its corresponding value contains a list of neighboring nodes. DFS uses the following data structures:

i. Graph: The graph is stored using a Python **dictionary**. Each node is associated with a list of its directly connected neighboring nodes.

ii. Stack: A Python **list** is used as a stack. It follows the **LIFO (Last In, First Out)** principle. The `pop()` operation removes the node from the top of the stack, while newly discovered nodes are added using `append()`.

iii. Visited: A Python **set** is used to store nodes that have already been discovered. It prevents the same node from being added to the stack multiple times and avoids unnecessary repeated traversal.

iv. Order: A Python **list** is used to maintain the order in which nodes are removed from the stack and visited during the search.

v. Start Node: The node from which the DFS traversal begins. In this program, the starting node is **A**.

vi. Goal Node: The node that the search attempts to find. In this program, goal node is **D**. The DFS process begins by placing the start node into the stack and marking it as visited. The node at the top of the stack is then removed and added to the visited order. If it is not the goal node, its unvisited neighboring nodes are added to the stack and marked as visited. The process continues until the goal node is found or the stack becomes empty. The use of a stack ensures that the search explores one branch as deeply as possible before backtracking, while the visited set prevents repeated processing of nodes.

Code:

```
GRAPH = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}

def depth_first_search(start: str, goal: str) -> tuple[bool, list[str]]:
    stack = [start]
    visited = {start}
    order = []

    while stack:
        node = stack.pop()
        order.append(node)

        if node == goal:
            return True, order
```

```

    for neighbor in GRAPH[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            stack.append(neighbor)

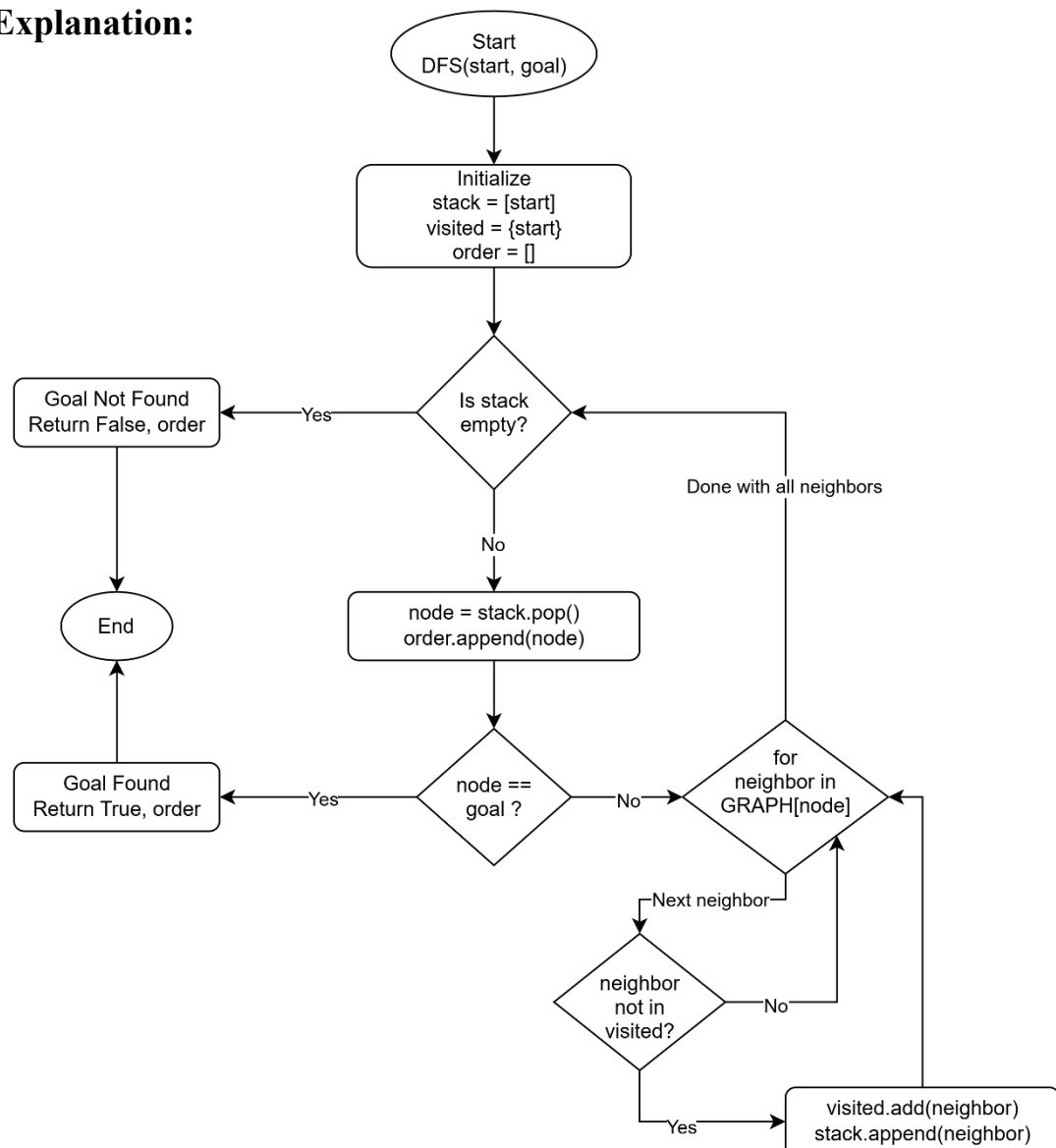
    return False, order

if __name__ == "__main__":
    status, path = depth_first_search("A", "D")

    print("Search Result:")
    print("\tStart : A")
    print("\tGoal : D")
    print("\tGoal Found :", status)
    print("\tVisited:", " -> ".join(path))

```

Code Explanation:



Output:

```
Search Result:  
  Start : A  
  Goal  : D  
  Goal Found : True  
  Visited: A -> C -> D
```

EXPERIMENT – 3

3. To implement the Constraint Satisfaction Problem (CSP) using Python with the backtracking algorithm.

Apparatus: Python

Theory: A Constraint Satisfaction Problem (CSP) is a problem in which a set of variables must be assigned values from their respective domains while satisfying a set of constraints.

A CSP consists of three main components:

1. **Variables:** The objects that need to be assigned values. In this program, the variables are A, B, C, and D.
2. **Domains:** The possible values that can be assigned to each variable. In this program, every variable has the domain [1, 2, 3].
3. **Constraints:** The restrictions that determine which assignments are valid.

In this implementation, the graph is represented using a dictionary, where each key represents a node and its corresponding value contains a list of neighboring nodes. The graph defines the relationships between variables.

The following constraints are applied:

- $A \neq B$
- $B \neq C$
- $A \neq C$
- $B < D$
- $C \neq D$

The program uses the following data structures:

i. Graph: The graph is stored using a Python dictionary. Each node is associated with a list of its directly connected neighboring nodes.

ii. Domains: A Python dictionary is used to store the possible values for each variable. Every variable has the domain [1, 2, 3].

iii. Solution: A Python dictionary is used to store the final assignment of values to the variables.

iv. Assignment: A Python dictionary is used to maintain the values assigned to variables during the search process.

v. Backtracking: Backtracking is a systematic search technique that assigns values to variables one by one. If an assignment violates any constraint, it is rejected. If no valid value is available, the algorithm backtracks to the previous variable and tries another value.

vi. Constraint Checking: The `is_valid()` function checks whether assigning a particular value to a variable satisfies all the constraints with the variables that have already been assigned.

Working Principle: The CSP process begins by creating a list of variables from the domain dictionary. An empty assignment dictionary is initialized to store the values assigned during the search. The backtracking function selects one variable at a time and tries each value from its domain. Before assigning a value, the `is_valid()` function checks whether the assignment satisfies all the given constraints. If the value is valid, it is added to the assignment dictionary and the algorithm recursively proceeds to the next variable. If a complete assignment is obtained, the search is successful. If a constraint is violated at any stage, the algorithm removes the assigned value and tries another value. The process continues until a valid solution is found or all possible assignments have been explored.

Code:

```

GRAPH = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}

DOMAINS = {
    "A": [1, 2, 3],
    "B": [1, 2, 3],
    "C": [1, 2, 3],
    "D": [1, 2, 3]
}

SOLUTION = {}

def constraint_satisfaction_problem() -> list[tuple[str, int]]:
    variables = list(DOMAINS)
    assignment = {}

    def is_valid(variable: str, value: int) -> bool:
        for neighbor in GRAPH[variable]:
            if neighbor not in assignment:
                continue

            if variable == "B" and neighbor == "D":
                if value >= assignment[neighbor]:
                    return False

            elif variable == "D" and neighbor == "B":
                if assignment[neighbor] >= value:
                    return False

            elif value == assignment[neighbor]:
                return False

        return True

    def backtrack(index: int) -> bool:
        if index == len(variables):
            return True

        variable = variables[index]

        for value in DOMAINS[variable]:
            if is_valid(variable, value):
                assignment[variable] = value

                if backtrack(index + 1):

```

```
        return True

    del assignment[variable]

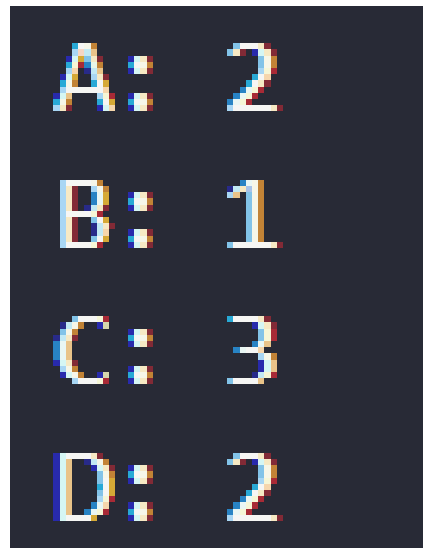
    return False

if backtrack(0):
    SOLUTION.update(assignment)
    return list(SOLUTION.items())

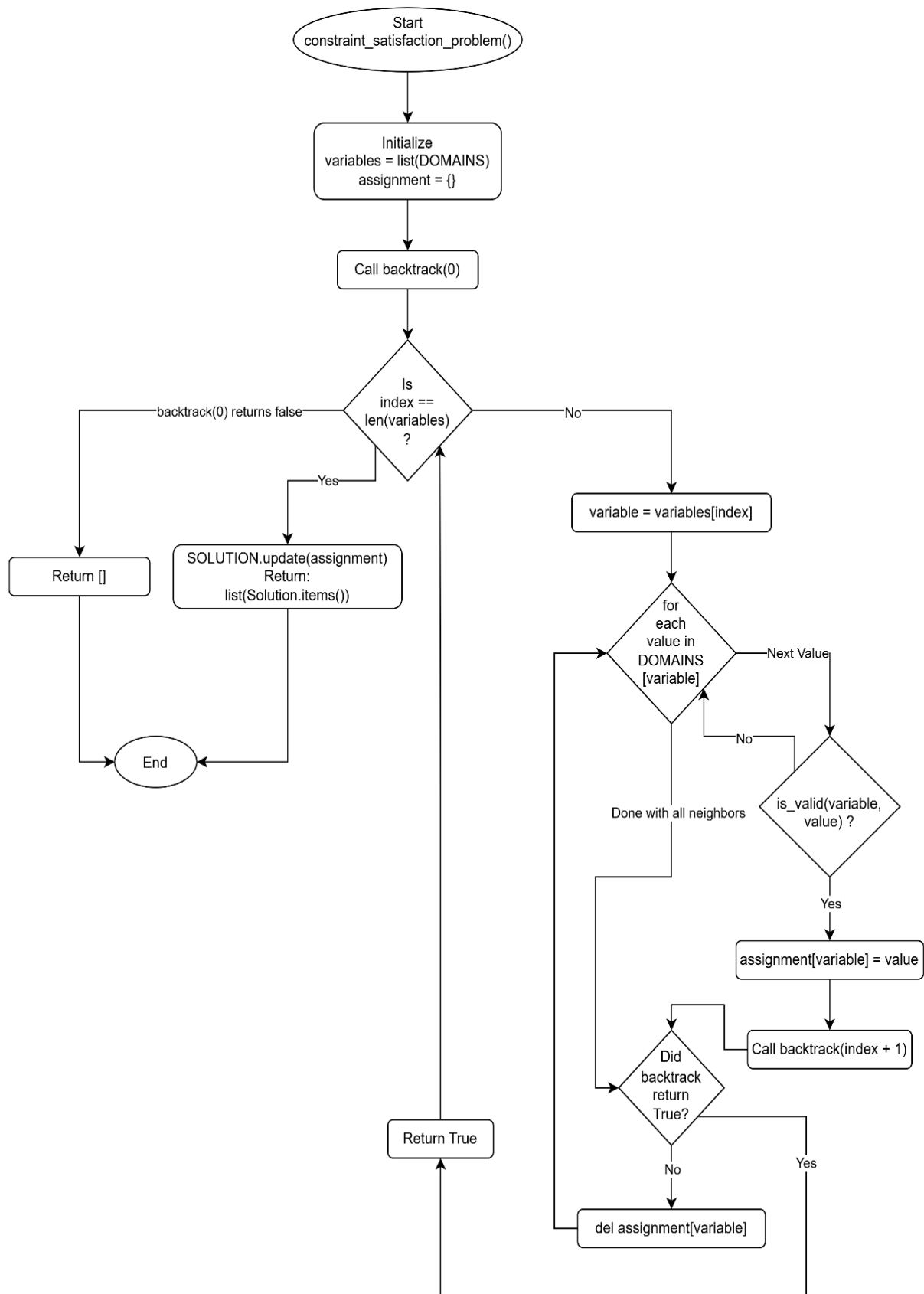
return []

if __name__ == "__main__":
    solution = constraint_satisfaction_problem()

    for variable, value in solution:
        print(f'{variable}: {value}')
```

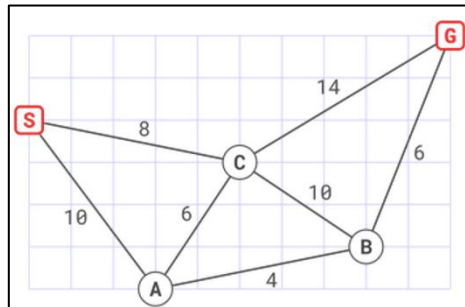
Output:

```
A: 2
B: 1
C: 3
D: 2
```

Code Explanation:

EXPERIMENT – 4

4. To implement the Hill Climbing search algorithm using Python to find a path from a start node to a goal node using Manhattan distance as the heuristic function.



Apparatus: Python

Theory: Hill Climbing is a heuristic search algorithm used in Artificial Intelligence to find a solution by continuously moving from the current state to a neighboring state that appears better according to a heuristic function. It is a local search algorithm because it considers the current state and its neighboring states rather than maintaining a complete search tree. In this implementation, the graph contains five nodes: S, A, C, B, and G. The objective is to find a path from the start node S to the goal node G.

Heuristic Function: The heuristic function used in this program is Manhattan distance. Manhattan distance calculates the distance between two points by adding the absolute differences of their x-coordinates and y-coordinates.

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

where:

$h(n)$ is the heuristic value of node n.

(x_n, y_n) are the coordinates of the current node.

(x_G, y_G) are the coordinates of the goal node G.

The graph is represented using a Python dictionary, where each key represents a node and its corresponding value contains a list of neighboring nodes. The graph is undirected, meaning movement is possible in both directions along the edges. The coordinates of the nodes are:

Node Coordinates Manhattan Distance

S	(0, 4)	12
A	(3, 0)	13
C	(5, 3)	8
B	(9, 1)	6
G	(11, 5)	0

Working Principle: The Hill Climbing process begins at the start node S. The algorithm examines all neighboring nodes and selects the neighbor having the lowest heuristic value. The selected neighbor is accepted only if its heuristic value is strictly smaller than the current node's heuristic value. If no neighbor provides an improvement, the algorithm stops because a local optimum has been reached.

The process continues until the goal node is reached or no better neighboring node is available.

Data Structures Used

- i. Graph: A Python dictionary storing nodes and their neighboring nodes.
- ii. Heuristic Dictionary: Stores the Manhattan distance of each node from the goal.
- iii. Path: A Python list storing the nodes visited during the search.
- iv. Current Node: Stores the node currently being explored.

Code:

```
def hill_climbing(graph, h, start, goal):
    current = start
    path = [current]

    print("=== STARTING HILL CLIMBING SEARCH (MANHATTAN) ===")

    while current != goal:
        neighbors = graph[current]

        if not neighbors:
            print("No neighbors found. Search failed.")
            return path

        best_neighbor = min(neighbors, key=lambda node: h[node])

        print(
            f"Current Node: {current} (h={h[current]}) "
            f"--> Best Neighbor: {best_neighbor} (h={h[best_neighbor]})"
        )

        if h[best_neighbor] >= h[current]:
            print("Local optimum reached. Halting search.")
            break

        current = best_neighbor
        path.append(current)

    if current == goal:
        print("Success! Goal reached.")
    else:
        print("Failed to reach the goal.")

    return path

if __name__ == "__main__":
    graph = {
        'S': ['C', 'A'],
        'A': ['S', 'C', 'B'],
        'C': ['S', 'A', 'B', 'G'],
        'B': ['A', 'C', 'G'],
        'G': ['C', 'B']
    }
```

```

h_manhattan = {
    'S': 12,      # |0-11| + |4-5| = 11 + 1 = 12
    'A': 13,      # |3-11| + |0-5| = 8 + 5 = 13
    'C': 8,       # |5-11| + |3-5| = 6 + 2 = 8
    'B': 6,       # |9-11| + |1-5| = 2 + 4 = 6
    'G': 0        # |11-11| + |5-5| = 0
}

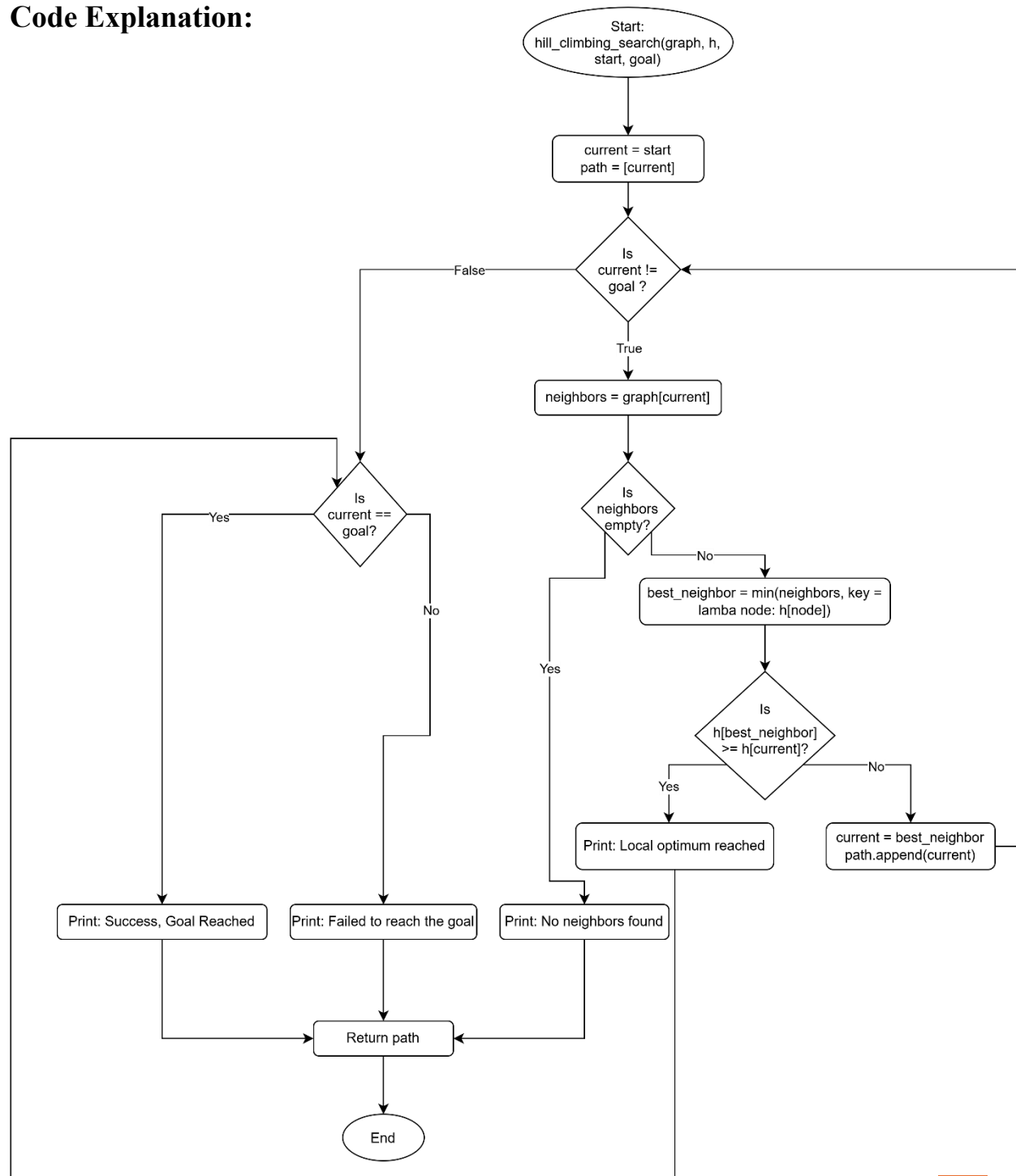
```

```

final_path = hill_climbing(graph, h_manhattan, start='S', goal='G')
print("Final Traversed Path:", final_path)

```

Code Explanation:



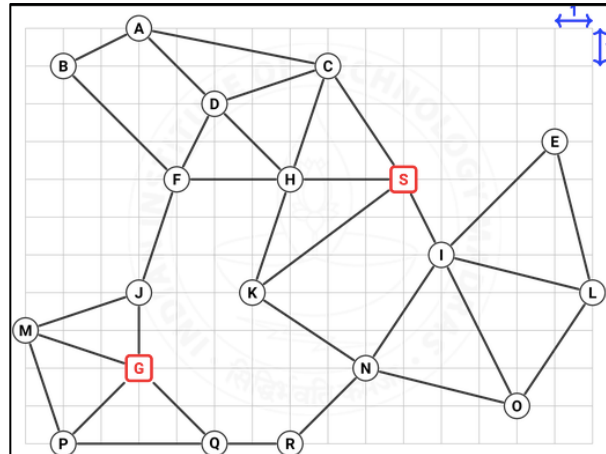
Output:

```

=== STARTING HILL CLIMBING SEARCH (MANHATTAN) ===
Current Node: S (h=12) --> Best Neighbor: C (h=8)
Current Node: C (h=8) --> Best Neighbor: G (h=0)
Success! Goal reached.
Final Traversed Path: ['S', 'C', 'G']

```

NOTE: Implementing the same hill climbing algorithm for a newer graph as given:

**Output:**

```

=== STARTING HILL CLIMBING SEARCH (MANHATTAN) ===
Current Node: S (h=12) --> Best Neighbor: K (h=5)
Current Node: K (h=5) --> Best Neighbor: N (h=6)
Local optimum reached. Halting search.
Failed to reach the goal.
Final Traversed Path: ['S', 'K']

```

EXPERIMENT – 5

5.